OBJECT ORIENTED PROGRAMMING

Prof. Dr. Muhammad Usman
Department of Computer Science
UET Mardan

Friend Functions

in C++

Granting selective access to private class members

What is a Friend Function?

A **friend function** is a function that is **NOT a member** (Non-member function) of a class, but has access to its **private** and **protected** members.

①

Declared inside class

Uses the friend keyword inside the class declaration

```
friend return-type function-name(arguments);
```

②

Defined outside class

Written like a normal function — no class scope needed

③

Not a member function

Has no this pointer. Called like a regular function

Syntax of Friend Function

C++

```
class Box {
private:
    double width;
public:
    Box(double w) : width(w) {}
    // friend declaration — inside class
    friend double getWidth(const Box& b);
};

// friend definition — outside class (no friend keyword)
double getWidth(const Box& b) {
    return b.width;   // accesses private member ✅
}
```

⚠ The friend keyword appears ONLY in the declaration inside the class — never in the definition outside.

Example 1 — Basic Friend Function

Code

```
#include <iostream>
using namespace std;
class Box {
private:
    double width;
public:
    Box(double w):width(w){}
    friend double getWidth(const Box& b);
};
double getWidth(const Box& b) {
    return b.width;
}
int main() {
    Box b(10.5);
    cout << getWidth(b);
}
```

How it works

1. Box stores width as private member
2. getWidth() declared as friend inside Box
3. getWidth() defined outside — no class scope
4. b.width accessible inside getWidth()
5. Called like a normal function in main()

Output: Width: 10.5

Example 2 — Friend of Two Classes

Code

```
class Distance;  // forward declaration
class Feet {
private:  int feet;
public:  Feet(int f):feet(f){}
    friend void addDistances(Feet, Distance);                    ← friend of Feet
};
class Distance {
private:  int meters;
public:  Distance(int m):meters(m){}
    friend void addDistances(Feet, Distance);                    ← friend of Distance
};
void addDistances(Feet f, Distance d) {
    cout << f.feet + d.meters;  // accesses both ✅
}
```

💡 A single function can be a friend of MULTIPLE classes — giving it access to all their private members.

Example 3 [1/2]

frend.cpp

```
2   // Friends can access private members of a class.
3   #include <iostream>
4
5   using std::cout;
6   using std::endl;
7
8   // Count class definition
9   class Count {
10    friend void setX( Count &, int ); // friend declaration
11
12  public:
13
14    // constructor
15    Count()
16       : x( 0 )  // initialize x to 0
17    {
18       // empty body
19
20  } // end Count constructor
21  // output x
23   void print() const
24    {
25       cout << x << endl;
26
27    } // end function print
28
29    private:
30       int x;  // data member
31
32  }; // end class Count
33
34    // function setX can modify private data of Count
35    // because setX is declared as a friend of Count
36    void setX( Count &c, int val )
37    {
38       c.x = val;  // legal: setX is a friend of
Count
39
40    } // end function setX
```

Precede function prototype with keyword **`friend`**.

Pass **`Count`** object since C-style, standalone function.

Since **`setX`** **`friend`** of **`Count`**, can access and modify **`private`** data member **`x`**.

Example 3 [2/2]

frend.cpp

```
42    int main()
43    {
44       Count counter;       // create Count object
45
46       cout << "counter.x after i
47       counter.print();
48
49       setX( counter, 8 );  // set x with a friend
50
51       cout << "counter.x after call to setX friend function: ";
52       counter.print();
53
54       return 0;
55
56    } // end main
```

Use **friend** function to access and modify **private** data member **x**.

```
counter.x after instantiation: 0
counter.x after call to setX
friend function: 8
```

Pass by Value vs Pass by Const Reference

Pass by Value

```
double getWidth(Box b)
```

- Creates a full COPY of the object
- Copy constructor is called
- Extra memory used
- Slower for large objects
- Copy can be modified inside
- Original always safe

✕ Not Preferred

Pass by Const Reference

```
double getWidth(const Box& b)
```

- NO copy — uses original object
- No copy constructor call
- Only 8 bytes (address) passed
- Fast — even for large objects
- const prevents modification
- Original safe + enforced

✓ Best Practice

Rules & Characteristics

Declared inside the class using the friend keyword

Defined outside the class like a normal function

Can access private and protected members of the class

A function can be a friend of more than one class

NOT a member function — called without object (no dot operator)

Has NO this pointer — object must be passed as parameter

friend keyword NOT used in the definition outside the class

Friendship is not mutual — must be declared in both classes

Friendship is not inherited — derived classes don't inherit it

PROPERTIES OF FRIENDSHIP

PROPERTIES OF FRIENDSHIP

- Friendship granted, not taken
  - ❑ Class **B friend** of class **A**

  Class **A** must explicitly declare class **B friend**
- Not symmetric
  - ❑ Class **B friend** of class **A**
  - ❑ Class **A** not necessarily **friend** of class **B**
- Not transitive
  - ❑ Class **A friend** of class **B**
  - ❑ Class **B friend** of class **C**
  - ❑ Class **A** not necessarily **friend** of Class **C**

When to Use Friend Functions

01

Operator Overloading

When overloading << or >> operators to work with cout/cin, a friend function is the standard approach.

02

Two-Class Operations

When a function needs to access private data of two different classes simultaneously.

03

Helper Utilities

When a utility function logically belongs outside the class but still needs private data access.

04

Testing & Debugging

When writing unit tests that need to inspect the internal state of an object without public getters.

Summary

What	A non-member function with access to private/protected members
How	Declared with friend keyword inside class; defined outside normally
Key	No this pointer — object must be passed as parameter
Best	Always pass by const reference for efficiency and safety
When	Operator overloading, two-class operations, utilities, testing
Avoid	Overusing — breaks encapsulation; use only when truly needed

Friend Classes

in C++

Granting an entire class access to another class's private members

What is a Friend Class?

A **friend class** is a class whose **ALL member functions** can access the **private** and **protected** members of another class.

Friend Function vs Friend Class

Friend Function

- ONE specific function gets access
- Only that function can see private members
- Declared: friend void func();
- Fine-grained — selective access

Friend Class

- ENTIRE class gets access
- ALL its methods can see private members
- Declared: friend class ClassName;
- Broad — whole-class access

Friend Class

For example, to declare class **Teacher** as friend of class **Student**, place the following declaration in the definition of class **Student**

```
friend class Teacher;
```

All member functions of class **Teacher** will become friends of class **Student**

We do not use scope resolution operator (::) and class name with friend function

Syntax of Friend Class

Code

```
class ClassB;   // forward declaration

class ClassA {
private:
    int secretData;
public:
    ClassA(int d) : secretData(d) {}
    // declare ClassB as a friend
    friend class ClassB;
};

class ClassB {
public:
    void showSecret(ClassA& a) {
        cout << a.secretData; // ✅ OK
    }
};
```

Key Syntax Rules

① Use friend class keyword inside ClassA

② No friend keyword needed in ClassB definition

③ ClassB must receive ClassA object as parameter

④ ClassA object passed — ClassB has no this for ClassA

⑤ Friendship is NOT mutual — ClassA cannot access ClassB private

⚠ Friendship is NOT mutual. If ClassB is a friend of ClassA, ClassA is NOT automatically a friend of ClassB.

Example 1 — Car Inspector accessing Engine internals

Code

```
class Inspector;   // forward declaration
class Engine {
private:
    int horsepower;
    float temperature;
public:
    Engine(int hp, float t)
        : horsepower(hp), temperature(t) {}
    friend class Inspector;   // ← friend class
};

class Inspector {
public:
    void checkHP(Engine& e) {
        cout << e.horsepower;  // ✅
    }
    void checkTemp(Engine& e) {
        cout << e.temperature; // ✅
    }
};
```

How It Works

Engine Class

- horsepower (private)
- temperature (private)

friend class Inspector

Inspector Class

checkHP() — sees horsepower ✓
checkTemp() — sees temperature ✓

```
Engine e(200, 85.5);
Inspector i;
i.checkHP(e);
i.checkTemp(e);
```

Output: 200 85.5

Example 1 — Complete Program & Output

main.cpp

```cpp
#include <iostream>
using namespace std;
class Inspector;
class Engine {
private:  int hp;  float temp;
public:
    Engine(int h, float t):hp(h),temp(t){}
    friend class Inspector;
};
class Inspector {
public:
    void checkHP(Engine& e) {
        cout<<"HP: "<<e.hp<<endl;
    }
    void checkTemp(Engine& e) {
        cout<<"Temp: "<<e.temp<<endl;
    }
};
int main() {
    Engine e(200, 85.5);
    Inspector i;
    i.checkHP(e);
    i.checkTemp(e);
}
```

Output

```
HP: 200
Temp: 85.5
```

Execution Steps

1. Engine e(200, 85.5) created
   hp=200, temp=85.5 stored privately
2. Inspector i created
   (no data — only methods)
3. i.checkHP(e) called
   Accesses e.hp directly — allowed ✓
4. i.checkTemp(e) called
   Accesses e.temp directly — allowed ✓

Example 2 — Auditor class inspecting BankAccount

Code

```
#include <iostream>
#include <string>
using namespace std;
class Auditor;    // forward declaration
class BankAccount {
private:
    string owner;
    double balance;
    int accNumber;
public:
    BankAccount(string o,double b,int n)
        :owner(o),balance(b),accNumber(n){}
    friend class Auditor;    // ← friend class
};
class Auditor {
public:
    void showOwner(BankAccount& a) {
        cout<<"Owner: "<<a.owner<<endl; }
    }
    void showBalance(BankAccount& a) {
        cout<<"Balance: "<<a.balance<<endl;
    }
    void fullAudit(BankAccount& a) {
        cout<<a.accNumber<<" "
        <<a.owner<<" "
        <<a.balance;
    }
};
```

Class Diagram

BankAccount

- owner (private)
- balance (private)
- accNumber (private)

friend class Auditor;

Auditor

+ showOwner() ✓
+ showBalance() ✓
+ fullAudit() ✓

ALL 3 methods of Auditor can access ALL 3 private members of BankAccount

Example 2 — main() Usage & Output

main.cpp

```
int main() {
    // create a BankAccount
    BankAccount acc("Ali", 50000.0, 1001);

    // create Auditor (friend class)
    Auditor aud;

    // access all private members
    aud.showOwner(acc);  // accesses owner
    aud.showBalance(acc); // accesses balance
    aud.fullAudit(acc);  // accesses all 3
    return 0;
}
```

Output

```
Owner: Ali
Balance: 50000
1001 Ali 50000
```

Key Observations

- ✓ All 3 Auditor methods access ALL private members of BankAccount
- ✓ No getters/setters needed — direct access via friendship
- ✓ BankAccount object passed as parameter to each method
- ✗ BankAccount CANNOT access Auditor's private members — NOT mutual
- ✗ Derived classes of Auditor do NOT automatically inherit friendship
- ⚠ Use friend class sparingly — breaks encapsulation if overused

Summary — Friend Classes

What	A class declared as friend can access ALL private and protected members of another class
Declare	Use: friend class ClassName; inside the class granting access
No mutual	Friendship is NOT mutual. ClassB being friend of ClassA does NOT make ClassA friend of ClassB
No inherit	Friend class relationship is NOT inherited by derived classes
Ex 1	Inspector class — all its methods freely inspect Engine's private horsepower and temperature
Ex 2	Auditor class — all its methods freely inspect BankAccount's private owner, balance, accNumber

Friend Function vs Friend Class — Full Comparison

Feature	Friend Function	Friend Class
What gets access	ONE specific function	ALL methods of the class
Keyword used	friend void func();	friend class ClassName;
Access scope	Fine-grained / selective	Broad / whole-class
Mutual by default	✕ No	✕ No
Inherited	✕ Not inherited	✕ Not inherited
Has this pointer	✕ No (non-member)	☑ Yes (own class only)
Object passed as	Parameter	Parameter
Best used for	Operator overloading, utilities	Tightly coupled class pairs
Encapsulation impact	Minor break	Larger break — use sparingly